

Teaching Exam

**Subject –Computer
Science**

Introduction to Python



Lecture No.- 02

By-Satyendra Chaudhary Sir

Topics to be Covered



Topic

→ Class / Objects

→ Abstract classes

Object Oriented Programming is a way of computer programming using the idea of “objects” to represents data and methods. It is also, an approach used for creating neat and reusable code instead of a redundant one.

2. Difference between Object-Oriented and Procedural Oriented Programming

Object-Oriented Programming (OOP)	Procedural-Oriented Programming (Pop)
It is a bottom-up approach	It is a top-down approach
Program is divided into objects	Program is divided into functions
Makes use of Access modifiers 'public', 'private', 'protected'	Doesn't use Access modifiers
It is more secure	It is less secure
Object can move freely within member functions	Data can move freely from function to function within programs
It supports inheritance	It does not support inheritance

3. What are Classes and Objects?

A class is a collection of objects or you can say it is a blueprint of objects defining the common attributes and behavior. Now the question arises, how do you do that?

Class is defined under a "**Class**" Keyword.

Example:

```
class class1(): // class 1 is the name of the class
```

Creating an Object and Class in python:

Example:

```
class employee():  
    def __init__(self, name, age, id, salary): //creating a function  
        self.name = name // self is an instance of a class  
        self.age = age  
        self.salary = salary  
        self.id = id  
  
emp1 = employee("harshit", 22, 1000, 1234) //creating objects  
emp2 = employee("arjun", 23, 2000, 2234)  
print(emp1.__dict__) // Prints dictionary
```

emp1.names

4. Object-Oriented Programming methodologies:

- ☐ **Inheritance** ✓
- ☐ **Polymorphism** ✓
- ☐ **Encapsulation** ✓
- ☐ **Abstraction** ✓

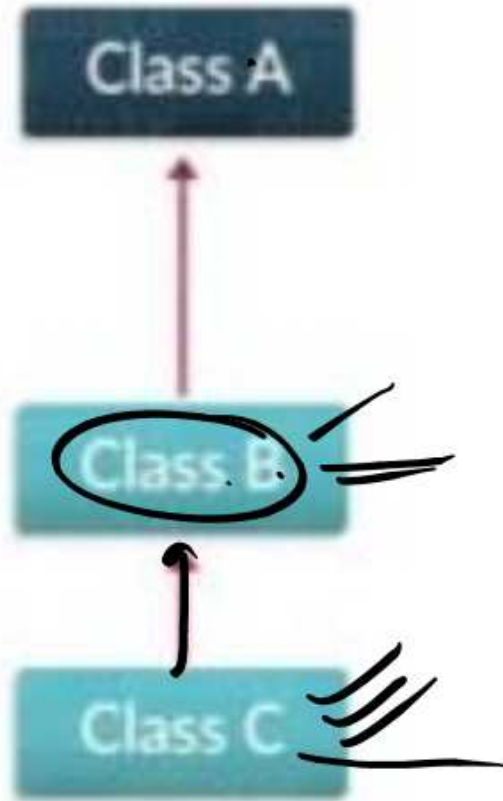
Inheritance:

Ever heard of this dialogue from relatives “you look exactly like your father/mother” the reason behind this is called ‘inheritance’. From the Programming aspect, It generally means “inheriting or transfer of characteristics from parent to child class without any modification”. The new class is called the derived/child class and the one from which it is derived is called a parent/base class.

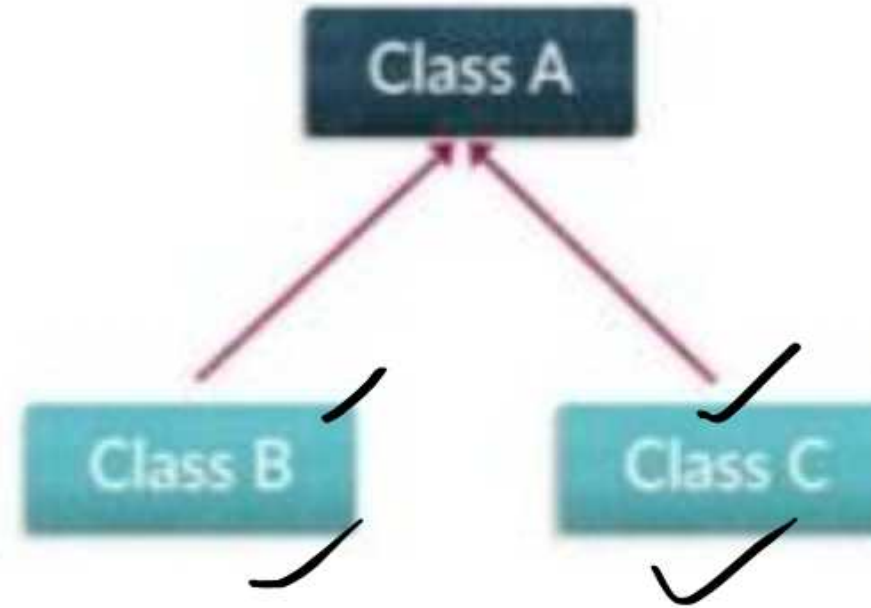
Types Of Inheritance



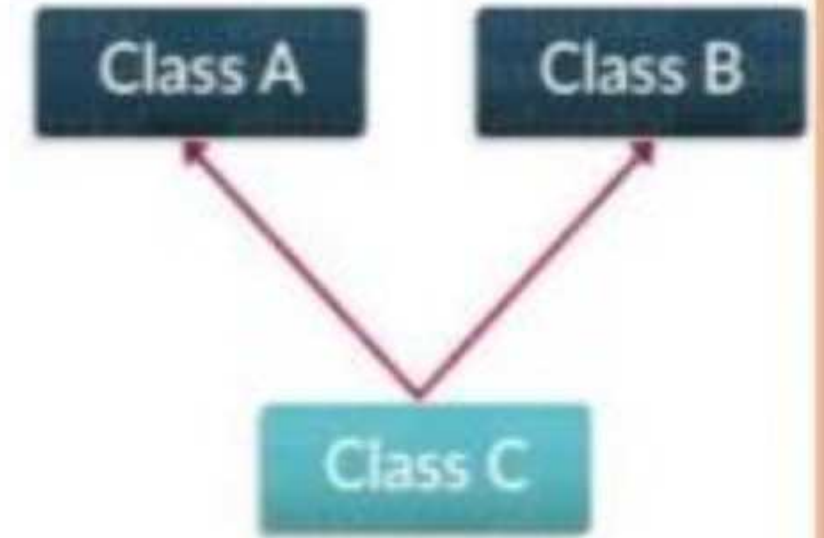
Single Inheritance



Multilevel Inheritance



Hierarchical Inheritance



Multiple Inheritance

Single Inheritance:

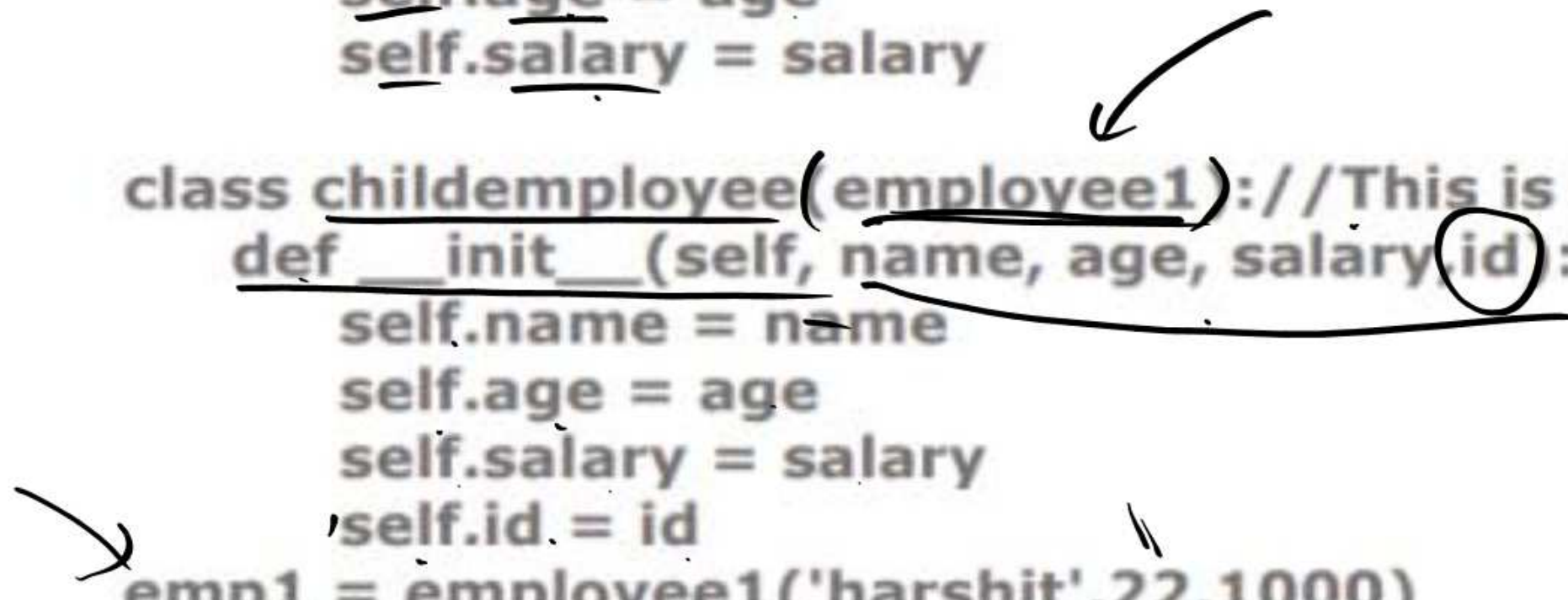
Single level inheritance enables a derived class to inherit characteristics from a single parent class.

Example:

```
class employee1(): // This is a parent class
    def __init__(self, name, age, salary):
        self.name = name
        self.age = age
        self.salary = salary

class childemployee(employee1): // This is a child class
    def __init__(self, name, age, salary, id):
        self.name = name
        self.age = age
        self.salary = salary
        self.id = id

emp1 = employee1('harshit', 22, 1000)
print(emp1.age)
```



Output: 22

Multilevel Inheritance:

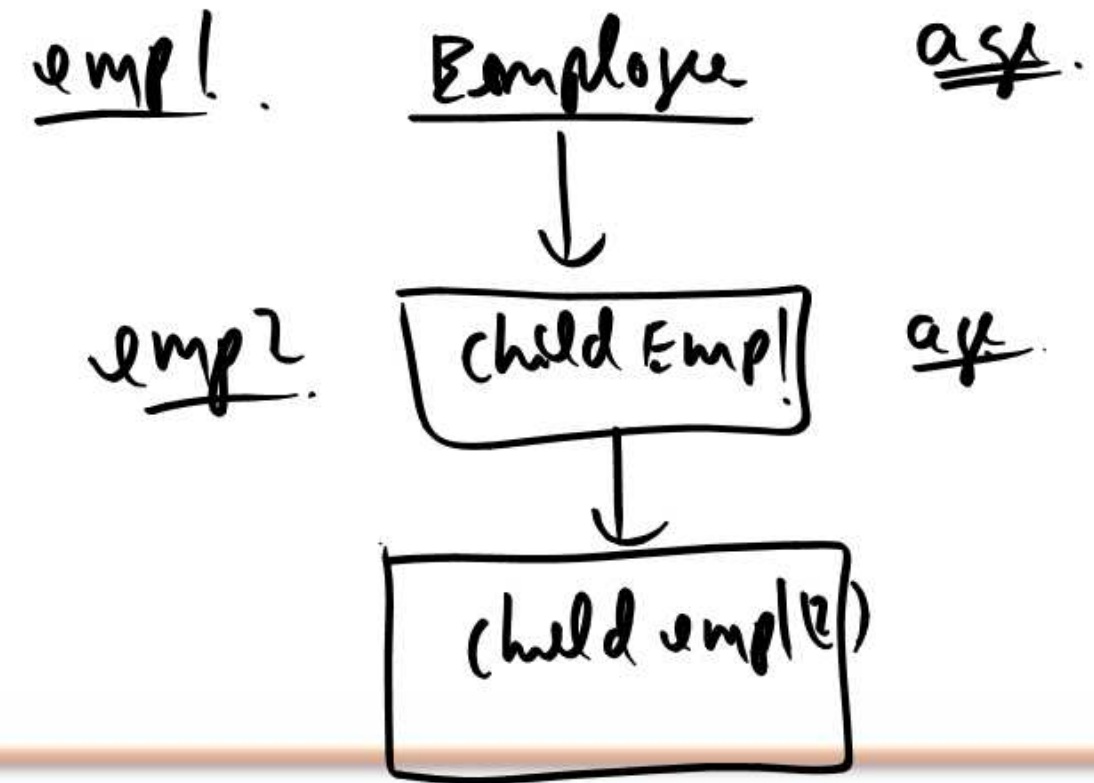
Multi-level inheritance enables a derived class to inherit properties from an immediate parent class which in turn inherits properties from his parent class.

Example:

```
class employee(): // Super class
    def __init__(self, name, age, salary):
        self.name = name
        self.age = age
        self.salary = salary
class childemployee1(employee): // First child class
    def __init__(self, name, age, salary):
        self.name = name
        self.age = age
        self.salary = salary
```



```
class childemployee2(childemployee1): // Second child class
    def __init__(self, name, age, salary):
        self.name = name
        self.age = age
        self.salary = salary
emp1 = employee('harshit', 22, 1000)
emp2 = childemployee1('arjun', 23, 2000)
print(emp1.age)
print(emp2.age)
```



Output: 22,23

Hierarchical Inheritance:

Hierarchical level inheritance enables more than one derived class to inherit properties from a parent class.

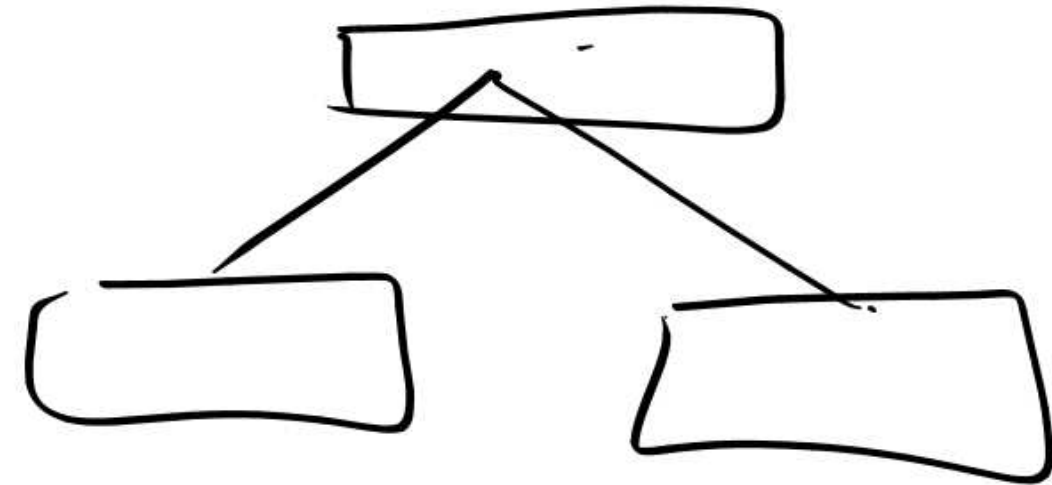
Example:

```
class employee():  
    def __init__(self, name, age, salary):    //Hierarchical Inheritance  
        self.name = name  
        self.age = age  
        self.salary = salary
```



```
class childemployee1(employee):  
    def __init__(self, name, age, salary):  
        self.name = name  
        self.age = age  
        self.salary = salary
```

```
class childemployee2(employee):  
    def __init__(self, name, age, salary):  
        self.name = name  
        self.age = age  
        self.salary = salary  
emp1 = employee('harshit', 22, 1000)  
emp2 = employee('arjun', 23, 2000)
```



Multiple Inheritance:

Multiple level inheritance enables one derived class to inherit properties from more than one base class.

Example:

```
class employee1(): //Parent class  
    def __init__(self, name, age, salary):  
        self.name = name  
        self.age = age  
        self.salary = salary
```



```
class employee2() //Parent class
    def __init__(self,name,age,salary,id):
        self.name = name
        self.age = age
        self.salary = salary
        self.id = id

class childemployee(employee1,employee2):
    def __init__(self, name, age, salary,id):
        self.name = name
        self.age = age
        self.salary = salary
        self.id = id

emp1 = employee1('harshit',22,1000)
emp2 = employee2('arjun',23,2000,1234)
```

```
class Animal( ): # parent class /
```

```
    def __init__(self, name, age):
```

```
        self.name = name.  
        self.age = age.
```

```
    def speak(self):
```

```
        print ( "{self.name} is speaking" )
```

```
class Dog(Animal):
```

```
    def __init__(self, name, age, breed):
```

```
        super.__init__(name, age)
```

```
        self.breed = breed
```



```
def bark(self):  
    print(f"{self.name} of age {self.age} is barking")
```

```
my_Dog = Dog("Buddy", 5, "Bulldog")
```

my_Dog.speak()

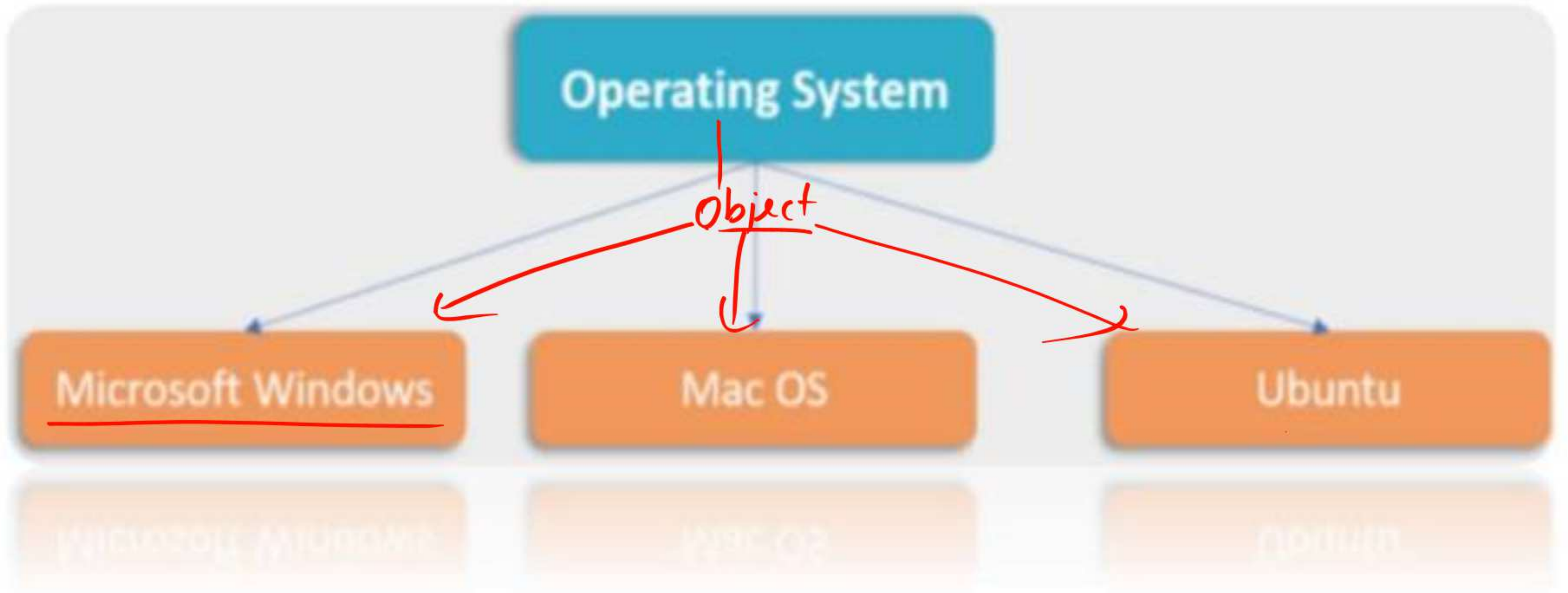
my_Dog.bark()

Buddy is speaker.

Buddy of age 5 is barking.

Polymorphism:

You all must have used GPS for navigating the route, Isn't it amazing how many different routes you come across for the same destination depending on the traffic, from a programming point of view this is called 'polymorphism'. It is one such OOP methodology where one task can be performed in several different ways. To put it in simple words, it is a property of an object which allows it to take multiple forms.



class
{ speak (name)
speak (age, name)
speak (— — i } }

Polymorphism is of two types:

❑ Compile-time Polymorphism

overloading
Python X

❑ Run-time Polymorphism

overriding

Compile-time Polymorphism:

A compile-time polymorphism also called as static polymorphism which gets resolved during the compilation time of the program. One common example is "method overloading"

Example:

```
class employee1():  
    def name(self):  
        print("Harshit is his name")  
    def salary(self):  
        print("3000 is his salary")  
    def age(self):  
        print("22 is his age")
```

```
class employee2():  
    def name(self):  
        print("Rahul is his name")  
    def salary(self):  
        print("4000 is his salary")  
    def age(self):  
        print("23 is his age")
```



```
def func(obj): // Method Overloading  
    obj.name()  
    obj.salary()  
    obj.age()  
  
obj_emp1 = employee1()  
obj_emp2 = employee2()  
func(obj_emp1)  
func(obj_emp2)
```

Output:

Harshit is his name
3000 is his salary
22 is his age
Rahul is his name
4000 is his salary
23 is his age

Python

Speak! !

Run-time Polymorphism:

A run-time Polymorphism is also, called as dynamic polymorphism where it gets resolved into the run time. One common example of Run-time polymorphism is "method overriding".

Example:

```
class employee():  
    def __init__(self,name,age,id,salary):  
        self.name = name  
        self.age = age  
        self.salary = salary  
        self.id = id  
    def earn(self):  
        pass  
  
class childemployee1(employee):  
    def earn(self): //Run-time polymorphism  
        print("no money")
```

```
class childemployee2(employee):  
    def earn(self):  
        print("has money")
```

```
c = childemployee1  
c.earn(employee)  
d = childemployee2  
d.earn(employee)
```

Output: no money, has money


```
1 # Base class
2 class Animal:
3     def make_sound(self):
4         print("Some generic animal sound")
5
6 # Derived class 1
7 class Dog(Animal):
8     def make_sound(self):
9         print("Bark! Bark!")
10
11 # Derived class 2
12 class Cat(Animal):
13     def make_sound(self):
14         print("Meow! Meow!")
15
16 # Function that uses polymorphism
17 def animal_sound(animal: Animal):
18     """
19     Accepts any object of type Animal (or its subclass)
20     and calls its make_sound() method.
21     """
```

```
21 """
22 animal.make_sound()
23
24 # Create objects of derived classes
25 dog = Dog()
26 cat = Cat()
27
28 # Call the polymorphic function
29 animal_sound(dog) # Output: Bark! Bark!
30 animal_sound(cat) # Output: Meow! Meow!
31
```

`__init__` Method – *The Constructor*

✔ What is it?

`__init__` is a special method in Python used to initialize (construct) an object when it's created from a class.

🧠 Think of it as:

The method that runs **automatically** when you create a new object of the class.

`__getitem__` and `__setitem__` – *Custom Indexing*

These are **magic methods** (also called **dunder methods**) that allow you to make your custom objects behave like Python's built-in lists or dictionaries.

◆ `__getitem__(self, key)`

Used to retrieve an item using square brackets.

◆ `__setitem__(self, key, value)`

Used to set an item using square brackets.


```
class Student:
    def __init__(self, name, age):
        # __init__ sets up student info and an empty marks dictionary
        self.name = name
        self.age = age
        self.marks = {} # Will store subject: marks pairs
        # Dictionary
    def __getitem__(self, subject): # key
        # Enables: student["Math"]
        return self.marks.get(subject, "Subject not found")
    def __setitem__(self, subject, score):
        # Enables: student["Math"] = 95
        self.marks[subject] = score
    def average(self):
        # Calculates the average of all marks
        if not self.marks:
            return 0
        return sum(self.marks.values()) / len(self.marks)
```

[]
()
{ }

```
def display_info(self):  
    print(f"Name: {self.name}, Age: {self.age}")  
    print("Marks:")  
    for subject, score in self.marks.items():  
        print(f" {subject}: {score}")  
    print(f"Average Marks: {self.average():.2f}")
```

Create a student

```
s1 = Student("Satyendra", 36)
```

Set marks using __setitem__

```
→ s1["Math"] = 95  
s1["Science"] = 88  
s1["English"] = 92
```

Access marks using __getitem__

```
print(s1["Math"]) # Output: 95  
print(s1["History"]) # Output: Subject not found
```

Display full student info

```
s1.display_info()
```

Output:

Name: Satyendra, Age: 36

Marks:

Math: 95

Science: 88

English: 92

Average Marks: 91.67



<code>__init__</code>	Initializing name, age, and marks
Class	<code>Student</code> with data and behavior
<code>__getitem__</code>	Get marks using square brackets
<code>__setitem__</code>	Set marks using square brackets
Custom Method	<code>average()</code> to compute average marks
Real-world structure	Acts like a real student record system

What is the primary goal of Object-Oriented Programming (OOP)?

- ☒ (A) Code reusability and modularity ✓
- ☐ (B) Efficient memory utilization ✗
- ☐ (C) Procedural abstraction ✗
- ☐ (D) Dynamic typing ✗

In Python, what is a class?

- (A) A function ✓
- (B) A module ✓
- (C) A code block ✓
- (D) A blueprint for creating objects ✓

What is an instance variable in a class?

- (A) A variable shared among all instances of a class
- (B) A variable defined outside the class
- (C) A variable specific to an instance of a class
- (D) A variable used for method overloading

What is encapsulation in OOP?

- (A) The process of converting a class into a module
- (B) Combining multiple classes into a single class X
- (C) Restricting access to certain components of an object and hiding the implementation details ✓
- (D) Creating a new instance of a class X



What is the difference between a class variable and an instance variable?

- (A) Class variables are specific to instances, while instance variables are shared among all instances. ✗
- (B) Class variables are defined outside the class, while instance variables are defined inside the class. ✗
- (C) ✓ Class variables are shared among all instances, while instance variables are specific to each instance. ✓
- (D) Class variables are used for method overloading, while instance variables are used for method overriding.

class

mark = 50

__init__()

Inheritance in OOP allows a class to:

- (A) Combine multiple classes into one
- (B) Define multiple methods with the same name
- (C) Inherit properties and behaviors from a superclass
- (D) Restrict access to certain components of an object

What is polymorphism in OOP?

- (A) Combining multiple classes into one
- (B) Restricting access to certain components of an object
- (C) Defining multiple methods with the same name in a class
- (D) Providing a single interface for different types

Which keyword is used to achieve method overloading in Python?

- ☐ (A) overload
- ☐ (B) method
- ☐ (C) override
- ☒ (D) Python does not support method overloading //

What is the primary purpose of abstraction in OOP?

abstract class

- (A) Combining multiple classes into one
- (B) Restricting access to certain components of an object
- (C) Defining multiple methods with the same name in a class
- (D) Simplifying complex systems by modeling classes based on their essential characteristics

What is the term for the process of creating a new class based on an existing class?

(A) Encapsulation

(B) Abstraction

(C) Inheritance

(D) Polymorphism

In OOP, what is the benefit of using encapsulation?

- (A) Improved code organization ✓
- (B) Code reusability ✓
- (C) Enhanced code readability ✓
- (D) Increased system performance ✗

How does polymorphism contribute to code flexibility in OOP?

- (A) By combining multiple classes into one
- (B) By providing a single interface for different types
- (C) ~~By restricting access to certain components of an object~~
- (D) By defining multiple methods with the same name

What is a metaclass in Python?

- ☐ A A class that inherits from multiple classes
- ☐ B A class for creating classes
- ☐ C A class that cannot be instantiated
- ☐ D A class that overrides all methods in a superclass

How does encapsulation contribute to data protection in OOP?

- (A) By exposing all class attributes publicly ✗
- (B) By allowing direct access to private attributes ✗
- (C) By restricting access to certain components of an object ✓
- (D) By promoting global access to class members ✗



2 mins Summary



TEACHING
WALLAH

Topic

LOOPS

THANK YOU